

Week 6: Indexes and Query Optimization

Phase 2: PostgreSQL Mastery | Week 6 of 12

Week Overview

This week teaches the most practical performance skill: understanding how PostgreSQL executes queries and how to make them faster. You will learn every index type, master EXPLAIN ANALYZE output, and develop an intuition for when PostgreSQL uses indexes vs. when it doesn't. By Friday you will be able to turn a 30-second query into a 30-millisecond one.

Focus: EXPLAIN ANALYZE is your most important tool. Use it before and after every optimization.

Learning Objectives

By the end of this week, you will be able to:

- Explain how B-Tree, Hash, GIN, GiST, BRIN, and SP-GiST indexes work.
 - Read and interpret EXPLAIN ANALYZE output including seq scans, index scans, and hash joins.
 - Use partial, covering, expression, and composite indexes correctly.
 - Recognize the 10 most common query performance anti-patterns.
 - Use `pg_stat_user_indexes` and `pg_stat_user_tables` to find unused indexes.
 - Tune `work_mem`, `effective_cache_size`, and `random_page_cost`.
 - Write queries that leverage index-only scans.
 - Use `CREATE INDEX CONCURRENTLY` for zero-downtime index creation.
-

Required Reading

- `07_Indexes/` — All files
 - `08_Query_Optimization/` — All files
-

Daily Schedule

Monday — Index Types and Internals (60 min)

Topics:

- B-Tree: default, ordered, supports =, <, >, BETWEEN, LIKE 'prefix%'
- Hash: only equality (=), faster than B-Tree for exact match
- GIN: inverted index for arrays, JSONB, full-text search
- GiST: geometric data, ranges, full-text (slower writes, flexible)
- BRIN: very fast, small, only for naturally ordered columns (timestamps, sequential IDs)
- SP-GiST: space partitioning (IP ranges, phone numbers)

```
-- Create a test table with 1 million rows
CREATE TABLE orders (
  id          BIGSERIAL PRIMARY KEY,
  customer_id INTEGER NOT NULL,
  status      VARCHAR(20) NOT NULL,
```

```

    amount          NUMERIC(12,2),
    created_at       TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    metadata         JSONB NOT NULL DEFAULT '{}',
    tags             TEXT[]
);

-- Generate test data
INSERT INTO orders (customer_id, status, amount, created_at, metadata, tags)
SELECT
    (RANDOM() * 10000)::INTEGER,
    (ARRAY['pending','completed','cancelled','refunded'])[1 + (RANDOM() *
3)::INTEGER],
    ROUND((RANDOM() * 1000 + 10)::NUMERIC, 2),
    NOW() - (RANDOM() * INTERVAL '365 days'),
    jsonb_build_object('region', (ARRAY['US','EU','APAC'])[1+(
RANDOM()*2)::INTEGER]),
    ARRAY[(ARRAY['web','mobile','api'])[1+(RANDOM()*2)::INTEGER]]
FROM generate_series(1, 1000000);

ANALYZE orders;

-- B-Tree (default)
CREATE INDEX idx_orders_customer ON orders (customer_id);
CREATE INDEX idx_orders_created ON orders (created_at);

-- Hash index (only equality)
CREATE INDEX idx_orders_status_hash ON orders USING HASH (status);

-- BRIN (timestamp is naturally ordered = great fit)
CREATE INDEX idx_orders_brin ON orders USING BRIN (created_at) WITH (pages_per_range
= 64);

-- GIN for JSONB
CREATE INDEX idx_orders_metadata ON orders USING GIN (metadata);

-- GIN for arrays
CREATE INDEX idx_orders_tags ON orders USING GIN (tags);

```

Tuesday — EXPLAIN ANALYZE Deep Dive (90 min)

Topics:

- EXPLAIN vs. EXPLAIN ANALYZE (ANALYZE actually runs the query)
- EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT) for full detail
- Seq Scan, Index Scan, Index Only Scan, Bitmap Index Scan
- Hash Join, Nested Loop Join, Merge Join — when each is chosen
- Understanding cost estimates: startup cost, total cost
- Actual vs. estimated rows (statistics accuracy)
- Using SET enable_seqscan = OFF to force index usage for testing

```

-- Compare: no index vs. with index
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM orders WHERE customer_id = 500;

-- Check if index is used
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM orders WHERE created_at >= NOW() - INTERVAL '7 days';

-- Function calls prevent index use
EXPLAIN SELECT * FROM orders WHERE LOWER(status) = 'completed'; -- Seq scan!
EXPLAIN SELECT * FROM orders WHERE status = 'completed';          -- Index scan

-- Index only scan (no heap access)
CREATE INDEX idx_orders_covering ON orders (customer_id, created_at)
    INCLUDE (amount, status);

EXPLAIN (ANALYZE, BUFFERS)
SELECT customer_id, created_at, amount, status
FROM orders WHERE customer_id = 500;
-- Should show "Index Only Scan"

-- Join analysis
EXPLAIN (ANALYZE, BUFFERS)
SELECT o.id, c.name, o.amount
FROM orders o
JOIN customers c ON o.customer_id = c.id
WHERE o.created_at >= NOW() - INTERVAL '30 days';

```

Wednesday — Advanced Index Strategies (90 min)

Topics:

- Partial indexes: reduce index size and false matches
- Expression indexes: index on function output
- Composite indexes: column order matters!
- Covering indexes with INCLUDE
- Index bloat and REINDEX
- Finding unused and duplicate indexes

```

-- Partial index: only active orders
CREATE INDEX idx_orders_pending ON orders (customer_id, created_at)
    WHERE status = 'pending';
-- Size: much smaller than full index

-- Expression index: enable LOWER() searches
CREATE INDEX idx_orders_status_lower ON orders (LOWER(status));
SELECT * FROM orders WHERE LOWER(status) = 'completed'; -- Now uses index

-- Composite index column order: (selectivity matters)
-- Rule: most selective column first, or order matching query WHERE + ORDER BY

```

```

CREATE INDEX idx_orders_cust_status ON orders (customer_id, status);
-- Supports: WHERE customer_id = X
--           WHERE customer_id = X AND status = Y
-- Does NOT efficiently support: WHERE status = Y (alone)

-- Finding unused indexes
SELECT schemaname, tablename, indexname,
       idx_scan, idx_tup_read, idx_tup_fetch
FROM pg_stat_user_indexes
ORDER BY idx_scan ASC;

-- Index sizes
SELECT indexname, pg_size_pretty(pg_relation_size(indexname::regclass)) AS size
FROM pg_stat_user_indexes
WHERE tablename = 'orders'
ORDER BY pg_relation_size(indexname::regclass) DESC;

-- Concurrent index creation (production safe)
CREATE INDEX CONCURRENTLY idx_orders_amount ON orders (amount);

```

Thursday — Common Anti-Patterns and Fixes (60 min)

Topics:

- 10 most common slow query patterns
- Implicit type casts killing indexes
- Leading wildcard LIKE problem
- OR vs. UNION ALL for better plans
- N+1 query patterns (application side)
- IN vs. ANY vs. = SOME

```

-- Anti-pattern 1: function on indexed column
-- BAD:
SELECT * FROM orders WHERE EXTRACT(YEAR FROM created_at) = 2024;
-- GOOD: range query uses index
SELECT * FROM orders WHERE created_at BETWEEN '2024-01-01' AND '2024-12-31';

-- Anti-pattern 2: implicit cast prevents index use
-- If customer_id is INTEGER but you pass a STRING:
SELECT * FROM orders WHERE customer_id = '500'; -- Type cast, may skip index
SELECT * FROM orders WHERE customer_id = 500;    -- Correct

-- Anti-pattern 3: OR can prevent good plans
-- BAD:
SELECT * FROM orders WHERE status = 'pending' OR status = 'refunded';
-- GOOD:
SELECT * FROM orders WHERE status IN ('pending', 'refunded');
-- OR EVEN BETTER when tables differ:
SELECT * FROM orders WHERE status = 'pending'
UNION ALL
SELECT * FROM orders WHERE status = 'refunded';

```

```

-- Anti-pattern 4: NOT IN with NULLs
-- NOT IN returns empty set if subquery contains any NULL
SELECT * FROM orders WHERE customer_id NOT IN (SELECT id FROM customers);
-- SAFE alternative:
SELECT * FROM orders o WHERE NOT EXISTS (
    SELECT 1 FROM customers c WHERE c.id = o.customer_id
);

-- Anti-pattern 5: SELECT * with large tables
-- SELECT * forces heap access even with covering indexes
-- Always list only needed columns

-- Performance configuration
SHOW work_mem;
SET work_mem = '256MB'; -- Per sort / hash operation
SHOW effective_cache_size;
SET effective_cache_size = '8GB'; -- Tell planner how much OS cache is available

```

Friday — Optimization Audit Mini-Project (45 min)

Mini-Project: Given a slow query set, optimize each one.

```

-- Slow query 1: Find all orders for a customer sorted by date
EXPLAIN ANALYZE
SELECT * FROM orders WHERE customer_id = 500 ORDER BY created_at DESC;
-- Task: identify the problem, add correct index, re-explain

-- Slow query 2: Monthly revenue report (no indexes on date)
EXPLAIN ANALYZE
SELECT DATE_TRUNC('month', created_at) AS month, SUM(amount)
FROM orders
WHERE created_at >= '2024-01-01'
GROUP BY 1 ORDER BY 1;
-- Task: what index helps? What about BRIN vs. B-Tree here?

-- Slow query 3: JSONB filter on metadata.region
EXPLAIN ANALYZE
SELECT COUNT(*) FROM orders WHERE metadata->>'region' = 'US';
-- Task: what index type is needed? Create it and verify

-- Slow query 4: Status-filtered count
EXPLAIN ANALYZE
SELECT COUNT(*) FROM orders WHERE status = 'pending';
-- Task: when is a partial index better than a full index here?

```

Practice Tasks

1. Use EXPLAIN ANALYZE to find which index a query is using.
2. Create a covering index that enables index-only scans for a frequent query.

3. Measure index creation time with and without CONCURRENTLY.
 4. Identify 3 unused indexes in your test database using `pg_stat_user_indexes`.
 5. Write a query that cannot use a B-Tree index and explain why.
 6. Convert a full-table scan query to use a partial index.
 7. Benchmark `IN` vs. `EXISTS` vs. `ANY` on a 100k row table.
 8. Set `work_mem` to 64MB and observe a sort changing from "disk" to "memory" sort.
-

Self-Assessment Checklist

- ☐ I can explain what each index type is best used for
 - ☐ I can read EXPLAIN ANALYZE output and identify bottlenecks
 - ☐ I created at least one partial index and measured its size vs. full index
 - ☐ I created a covering index that enables index-only scans
 - ☐ I know 5 common anti-patterns that prevent index use
 - ☐ I used `pg_stat_user_indexes` to check index usage
 - ☐ I understand composite index column ordering rules
-

Mock Interview Questions

1. What is the difference between a B-Tree and a GIN index?
 2. When would you use a partial index? Give a concrete example.
 3. What does "Seq Scan" vs. "Index Scan" mean in EXPLAIN output?
 4. Explain why `WHERE LOWER(email) = 'x'` might not use an index on `email`.
 5. What is a covering index and why does it eliminate heap access?
 6. When would you choose BRIN over B-Tree for a timestamp column?
 7. How do you add an index to a 100GB table without downtime?
 8. A query that was fast is now slow after the table grew. What's your process?
-

Resources

- This repo: `07_Indexes/` , `08_Query_Optimization/`
- pganalyze EXPLAIN tool: <https://explain.dalibo.com>
- pgtune: <https://pgtune.leopard.in.ua>